23

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

미들웨어

제2부 NestJS 핵심 · 6장 요청 파이프라인

모든 요청이 맨 먼저 지나는 관문, 맡길 일과 맡기지 않을 일



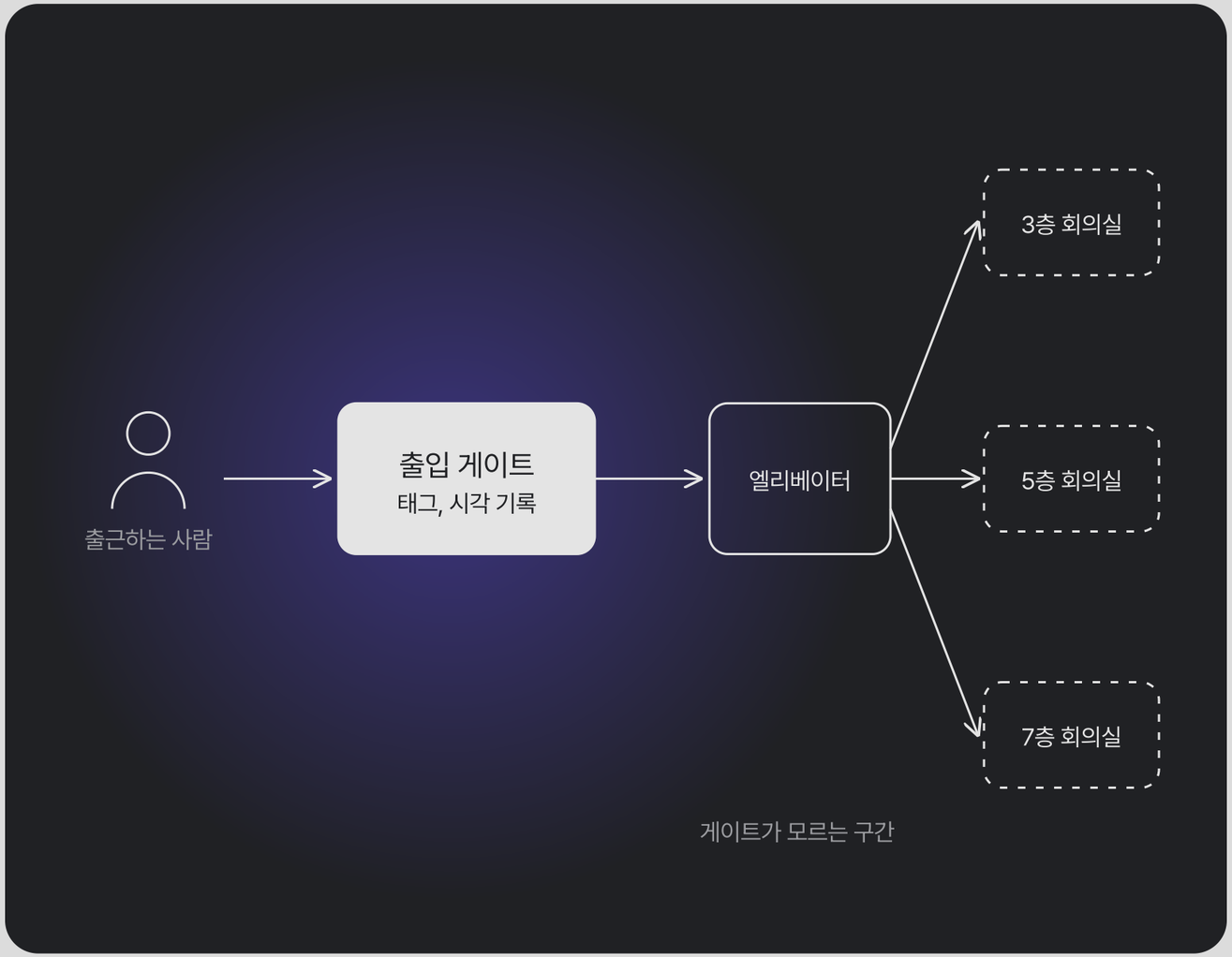
© 한상훈 NestJS 백엔드 마스터

건물 1층 출입 게이트

→ 몇 층에 가든 사원증 태그

→ 들어온 시각과 나간 시각 기록

→ 게이트가 모르는 것: 몇 층 어느 회의실로 가는지



© 한상훈 NestJS 백엔드 마스터

접근 로그에 없는 404

- → 가정: 핸들러마다 첫 줄에서 로그 기록
- → 고객 앱이 오타 난 주소로 요청 → 404, 로그 없음
- → 가드에 막힌 요청 → 403, 로그 없음
- → 핸들러까지 온 요청만 남는 기록
- → 가정: 회차 주소를 `performances` → `shows`로 변경, 고객 앱 한 화면은 옛 주소 호출 → 404만 쌓임
- → 접근 로그(access log): 들어온 요청을 전부 한 줄씩 남기는 기록

© 한상훈 NestJS 백엔드 마스터

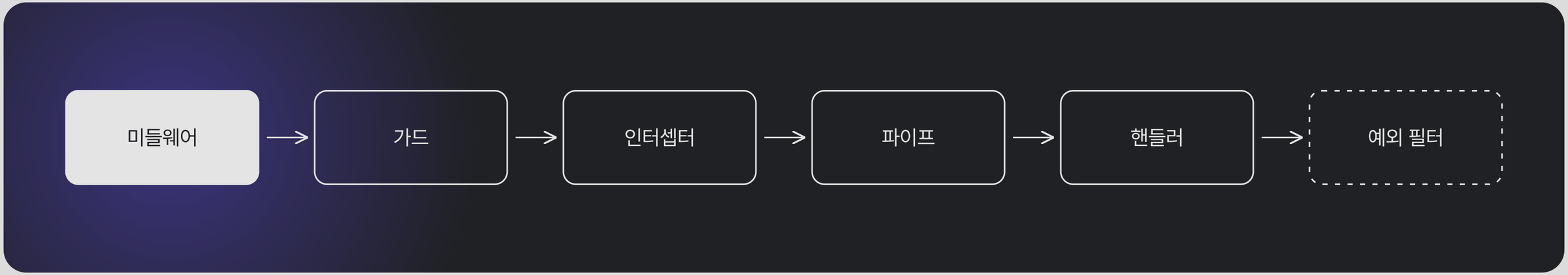
맨 앞의 관문

curl · 서버 로그

```
$ curl localhost:4022/nope
1 미들웨어 GET /nope
```

→ 라우트를 찾기 전에 실행

→ 없는 주소, 거절될 요청까지 모두 통과



© 한상훈 NestJS 백엔드 마스터

req, res, next

- → 미들웨어: 요청 객체, 응답 객체, `next` 함수를 받는 함수
- → Express 미들웨어와 같은 모양
- → `next()`: 다음 미들웨어나 라우트로 넘김
- → 응답을 직접 보내고 끝낼 수도 있음

© 한상훈 NestJS 백엔드 마스터

RequestLogMiddleware

request-log.middleware.ts

```ts
import { Injectable, Logger, NestMiddleware } from '@nestjs/common';
import type { NextFunction, Request, Response } from 'express';

@Injectable()
export class RequestLogMiddleware implements NestMiddleware {
  private readonly logger = new Logger('HTTP');

  use(req: Request, res: Response, next: NextFunction) {
    const started = performance.now();
    res.on('finish', () => {
      const ms = (performance.now() - started).toFixed(1);
      const id = req.headers['x-request-id'];
      this.logger.log(`${id} ${req.method} ${req.originalUrl} ${res.statusCode} ${ms}ms`);
    });
    next();
  }
}
```

© 한상훈 NestJS 백엔드 마스터

configure와 forRoutes

app.module.ts

```
@Module({ /* imports, controllers, providers 그대로 */ })
export class AppModule implements NestModule {
  configure(consumer: MiddlewareConsumer) {
    consumer.apply(RequestIdMiddleware, RequestLogMiddleware).forRoutes('*path');
  }
}
```

→ `apply`: 적용할 미들웨어, 적은 순서대로 실행

→ `forRoutes`: 적용할 경로나 컨트롤러, `'*path'`는 모든 경로

→ `exclude`: 빼고 싶은 경로

→ `forRoutes(컨트롤러 클래스)`: 그 컨트롤러 경로에만 (예: 결제 웹훅 서명 확인)

→ 기능 모듈마다 자기 configure 가능

© 한상훈 NestJS 백엔드 마스터

클래스 미들웨어와 함수 미들웨어

종류	모양	생성자 주입
클래스	`NestMiddleware`를 구현한 `@Injectable` 클래스	있음
함수	`(req, res, next) => { ... }`	없음

© 한상훈 NestJS 백엔드 마스터

응답이 끝난 뒤에 남기는 이유

- → `next()`를 부르는 순간: 상태 코드도, 걸린 시간도 아직 없음
- → `res.on('finish')`: 응답의 마지막 바이트를 보낸 뒤 실행
- → 그때 알 수 있는 것: 상태 코드, 들어온 뒤 걸린 시간
- → 출입 게이트: 나갈 때 한 번 더 찍어야 머문 시간이 나옴

© 한상훈 NestJS 백엔드 마스터

요청 로그

서버 로그

```
LOG [HTTP] deb5e0be-… POST /concerts 201 4.8ms
LOG [HTTP] 3e69d4c7-… POST /performances 201 0.5ms
LOG [HTTP] af68fab5-… POST /performances/1/seats 201 0.9ms
LOG [HTTP] 44995d59-… GET /performances/1/seats 200 0.7ms
LOG [HTTP] 514bc72f-… POST /performances/1/seats/A-9-9/hold 404 1.0ms
LOG [HTTP] 8c5d4098-… GET /nope 404 0.6ms
```

요청 ID는 앞 8자리만 표시

데이터 블록: 강사 PC 실측 0.3~7.9ms (Node v24.11.0, 2026-10-26), 시간은 환경마다 다름, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

404 두 줄

로그 줄	이유	핸들러 실행
`POST …/seats/A-9-9/hold 404`	좌석 A-9-9이 없음	실행됨
`GET /nope 404`	라우트가 없음	없음

→ 둘 다 미들웨어 로그에 남음

→ 하루치를 모으면: 상태 코드별 에러율, 경로별 처리 시간, 잘못 부르는 주소

© 한상훈 NestJS 백엔드 마스터

next()를 지운 코드

request-log.middleware.ts

```
use(req: Request, res: Response, next: NextFunction) {
  const started = performance.now();
  res.on('finish', () => { /* 로그 */ });
  // next();  <- 빠뜨림
}
```

© 한상훈 NestJS 백엔드 마스터

3초 뒤 시간 초과

curl

```
$ curl -m 3 localhost:4023/concerts
 [000] 3.010492s exit=28
```

→ 응답 없음, 상태 코드 없음

→ 서버 로그: 에러 없음, 요청 로그도 없음 (`finish`가 오지 않음)

→ 클라이언트가 정한 대기 시간이 끝나야 실패

© 한상훈 NestJS 백엔드 마스터

미들웨어가 지킬 것

→ 둘 중 하나는 반드시: `next()` 호출 또는 응답 전송

→ 둘 다 안 하면: 연결과 메모리를 쥔 채 멈춘 요청

→ 조건문 안에서만 `next()`를 부르는 코드: 다른 갈래 확인

→ 예: 앱 버전 헤더가 있으면 `next()`, 없으면 경고 로그만 → 옛 앱·외부 연동 요청만 전부 멈춤

© 한상훈 NestJS 백엔드 마스터

비동기 미들웨어의 예외

```
async use(req: Request, res: Response, next: NextFunction) {
  const on = await Promise.resolve(process.env.MAINTENANCE === 'on');
  if (on) throw new ServiceUnavailableException('점검 중입니다');
  next();
}
```

○ ○ ○ curl · 서버 로그

```
$ curl localhost:4023/concerts
{"type":"about:blank","title":"Service Unavailable","status":503,
 "detail":"점검 중입니다","instance":"/concerts"}
LOG [HTTP] 0b276d10-… GET /concerts 503 1.3ms
```

→ 미들웨어에서 던진 예외: 예외 필터가 받음

© 한상훈 NestJS 백엔드 마스터

결제가 안 됐다는 문의

→ 가정: 고객 문의 "10시 0분에 결제를 눌렀는데 결과가 안 나왔어요"

→ 그 1초의 로그: 수천 줄, 같은 경로·같은 상태 코드

→ 접근 로그: 개인 정보 없음, 로그인 전 요청은 이름도 없음

→ 찾을 열쇠: 요청마다 붙은 ID

→ 고객 앱이 응답 헤더의 ID를 오류 화면에 표시 → 고객센터가 ID로 검색

© 한상훈 NestJS 백엔드 마스터

RequestIdMiddleware

request-id.middleware.ts

```
@Injectable()
export class RequestIdMiddleware implements NestMiddleware {
  use(req: Request, res: Response, next: NextFunction) {
    const id = req.header('x-request-id') ?? randomUUID();
    req.headers['x-request-id'] = id;
    res.setHeader('x-request-id', id);
    next();
  }
}
```

→ 들어온 ID가 있으면 그대로, 없으면 새로 생성

→ 요청 헤더에 써 두면 뒤의 관문이 같은 ID를 읽음

© 한상훈 NestJS 백엔드 마스터

응답 헤더와 로그 줄

curl

```
$ curl -i localhost:4023/performances/1/seats
HTTP/1.1 200 OK
x-request-id: 44995d59-6b24-4013-b6b3-01d67d6f66bb
```

curl · 서버 로그

```
$ curl -H 'x-request-id: lb-7f3a91' localhost:4023/concerts/1
LOG [HTTP] lb-7f3a91 GET /concerts/1 200 0.3ms
```

→ 앞단(로드 밸런서)이 붙인 ID를 이어 씀 → 앞단 로그와 서버 로그를 같은 ID로 연결

© 한상훈 NestJS 백엔드 마스터

264KB 좌석 배치

curl · 서버 로그

```
$ wc -c layout.json
264515 layout.json
(A~H구역 × 40열 × 20석 = 6,400석)

$ curl -X POST localhost:4023/concerts --data-binary @layout.json
{"type":"about:blank","title":"Internal Server Error","status":500,
 "detail":"요청을 처리하지 못했습니다.","instance":"/concerts"}

ERROR [ProblemDetails] PayloadTooLargeError: request entity too large
  expected: 264515, length: 264515, limit: 102400
```

→ 요청 로그: 이 요청의 줄 없음

© 한상훈 NestJS 백엔드 마스터

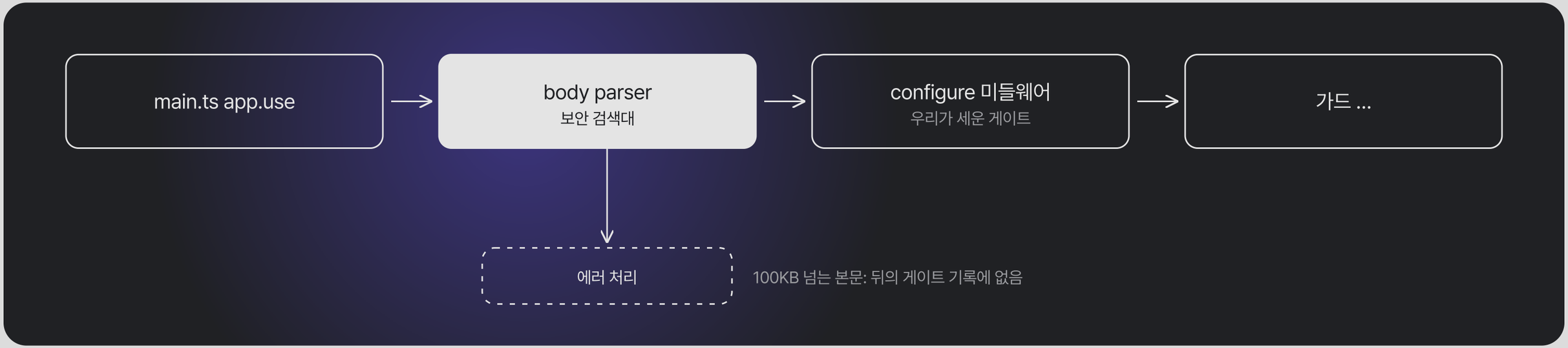
body parser도 미들웨어

→ Nest가 서버를 만들 때 JSON·urlencoded 본문 파서를 먼저 등록

→ 실행 순서: main.ts의 `app.use` → body parser → 모듈의 configure 미들웨어 → 가드 …

→ 기본 상한: 100KB(102,400바이트)

→ 파서에서 실패한 요청: 뒤의 미들웨어를 건너뛰고 바로 에러 처리



© 한상훈 NestJS 백엔드 마스터

413이어야 할 500

예외 필터	응답
Nest 기본 처리	413 `{"statusCode":413,"message":"request entity too large"}`
지금의 ProblemDetailsFilter	500, HttpException이 아니라서 모르는 에러로 처리

→ 상태 코드가 붙은 에러를 필터가 읽지 못함

© 한상훈 NestJS 백엔드 마스터

main.ts에서 먼저

main.ts

```ts
const app = await NestFactory.create(AppModule);
app.use(requestId, requestLog); // body parser보다 먼저
app.use(compression());
```

○ ○ ○ 서버 로그

```
LOG [HTTP] 4661e5d9-… POST /concerts 201 7.9ms
LOG [HTTP] ff788da5-… POST /concerts 500 2.5ms
LOG [HTTP] 1aa22630-… GET /nope 404 0.7ms
```

→ `requestId`·`requestLog`: 같은 일을 하는 함수 미들웨어로 바꿈, configure 등록은 삭제

© 한상훈 NestJS 백엔드 마스터

상한 바꾸기

main.ts

```ts
const app = await NestFactory.create<NestExpressApplication>(AppModule);
app.useBodyParser('json', { limit: '1mb' });
```

curl

```
$ curl -X POST localhost:4023/concerts --data-binary @layout.json
{"id":1,"title":"좌석 배치"}
```

→ 상한: 가장 큰 정상 요청에 맞춰 정함

→ 큰 파일: 본문 JSON 대신 업로드나 스트리밍

→ 이 설정은 시연만, 프로젝트는 기본 상한 유지

© 한상훈 NestJS 백엔드 마스터

미들웨어가 아는 것과 모르는 것

아는 것	모르는 것
메서드, 경로 문자열, 헤더	이 요청을 처리할 컨트롤러와 메서드
들어온 시각, 끝난 시각과 상태 코드	그 메서드에 붙은 표시(직원 전용 등)
요청 ID	파이프를 거친 인자 값

→ 맡길 일: 요청 로그, 요청 ID, 본문 파싱, 압축, 서버 전체 점검 중 차단

→ 특정 기능만 막는 점검: 핸들러를 알아야 하므로 다른 관문

→ 맡기지 않을 일: "이 핸들러를 이 사람이 써도 되나"

→ 경로 문자열로 권한 검사: 대문자, 끝 슬래시에서 검사와 라우팅이 엇갈림

© 한상훈 NestJS 백엔드 마스터

미들웨어

"미들웨어는 모든 요청이 지나는 입구라서, 어느 핸들러로 갈지 몰라도 되는 일만 맡깁니다."